

MR Drumset

A Mixed-Reality Percussion Instrument with Dataflow Audio Synthesis

Istrate Irina | Stancu Rares | Ioana Madalin | Jilavu Izabela | Manolachi George

April 2026

Abstract

MR Drumset is an interactive mixed-reality application that turns a Pico 4 headset into an eight-piece percussion instrument. Three independent processes cooperate over Socket.IO and OSC: a WebXR/Three.js frontend rendering the instruments and detecting stick collisions, a Node.js bridge forwarding hits as OSC messages, and a Pure Data patch that synthesises the sound using the dataflow paradigm central to this course. The frontend additionally doubles-renders on a desktop browser to produce a live spectator mirror of the VR session. This report describes the architecture, the swept-plane collision model used to get reliable hit detection on arbitrarily oriented surfaces, the role of visual dataflow programming in the audio engine, and the design trade-offs chosen to hit the 72 Hz performance budget of the target device.

Contents

1	Introduction	3
2	System Architecture	3
3	Frontend	4
3.1	Scene construction	4
3.2	Swept-plane collision	4
3.3	Particle system	5
3.4	Internal audio synthesis	5
4	Bridge Server	6
5	Pure Data Patch and the Dataflow Paradigm	6
5.1	Input and dispatch	6
5.2	Sample-based voicing	7
5.3	Master bus	8
5.4	Why dataflow matters here	8
6	Spectator Mirror	8
7	Building and Running	9
7.1	Prerequisites	9
7.2	One-time setup	9
7.3	Running the system	9
7.4	Verification	9
8	Performance Notes	9

9	Limitations and Future Work	10
10	Conclusion	10
	References	10

1 Introduction

The goal of MR Drumset is two-fold. First, to provide a physically believable drumset in virtual reality — realistic enough that a user holding two controllers can play it the way one plays a real kit, with the stick’s speed, approach angle, and impact location all mattering. Second, to demonstrate end-to-end integration between a modern WebXR stack and a *visual dataflow* audio-synthesis environment (Pure Data, colloquially Pd), which is the subject of the course this project is submitted for.

The user wears a Pico 4 headset, holds its two controllers as drumsticks, and plays the eight instruments arranged around them in a natural kit layout. Every strike produces three simultaneous effects:

1. an immediate visual flash, drum-head flex, and particle burst at the exact contact point;
2. an internal Web-Audio synthesiser voice triggered in the headset’s browser;
3. an OSC message dispatched over UDP to a Pure Data patch running on the host PC, which plays the corresponding drum recording — a three-layer velocity-crossfaded multisample for the bass, snare, and hi-hat, and a single recording scaled by velocity for the remaining drums.

The parallel Web-Audio and Pd engines are a deliberate choice: the browser provides low-latency sound in the headset for responsive feel, while Pd supplies the richer, live-tweakable engine that represents the dataflow component of the system.

2 System Architecture

The application is composed of three processes communicating over two channels (figure 1).

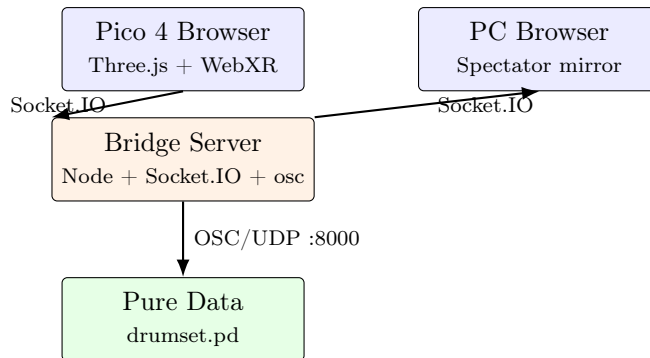


Figure 1: Process topology. The bridge is the only component with access to the OSC path; both browsers talk to it via Socket.IO.

Frontend. A single `main.js` is served by Vite (HTTPS via `basicSsl`) to *both* the headset and the desktop. It self-identifies its role at runtime: if the user presses the “Enter VR” button a WebXR session begins and the client becomes the producer; otherwise the client stays in spectator mode and consumes pose/hit messages broadcast to it by the bridge.

Bridge. A short Node program (`bridge/index.js`) runs a Socket.IO server on port 3000 and an OSC UDP sender targeting `127.0.0.1:8000`. Its responsibilities are symmetric: relay Socket.IO messages between browsers, and translate `drumHit` events into `/drum/hit` OSC packets for Pd.

Pure Data patch. `pd/drumset.pd` binds a UDP port, parses incoming OSC, dispatches to one of eight sample-playback voices by instrument name, and mixes them through a soft-limited stereo bus to the audio output. Voices are implemented as reusable abstractions (`voice-multi.pd` and `voice-single.pd`) instantiated with the drum’s name as a creation argument. The patch is the component that embodies the dataflow paradigm studied in this course — every box and cord is a first-class visual programming element.

3 Frontend

3.1 Scene construction

The scene is constructed with Three.js 0.160. The rendering pipeline uses PBR materials (`MeshStandardMaterial`) lit by a warm-on-cool four-point lighting rig and a pre-rendered environment map produced with `PMREMGGenerator` from the built-in `RoomEnvironment`. The environment map gives the metal cymbals and drum hoops realistic specular reflections at no runtime cost beyond a single setup pass.

Each of the eight instruments is built by a dedicated factory function that returns a Three.js group plus four collision descriptors:

```
{
    group,          // THREE.Group containing all visual parts
    hitMesh,        // primary surface      used for raycasting
    localHitCenter, // Vector3 in group-local space
    localHitNormal, // unit normal in group-local space
    hitRadius       // radius of the playable region
}
```

Three factory kinds cover the kit:

- **buildDrumTop:** a cylinder shell with top and bottom chrome hoops, a flat drum head disc, six-eight tensioning lugs, and an optional forward tilt (used by the rack and floor toms). The hit plane is the top head surface.
- **buildBassDrum:** a cylinder rotated onto its side so the player strikes the front head along the $+Z$ axis; legs support the shell, and the hit plane is the front-facing head.
- **buildCymbal** and **buildHiHat:** thin tapered discs with a central bell dome, tilted in two axes for natural angle. The hit plane is the upper disc surface.

The kit layout mirrors that of a real drumset: the bass drum sits on the floor in front of the player, the snare waist-high and slightly left, two rack toms mounted above the bass, the floor tom to the right, hi-hat to the left, crash upper-left and ride upper-right.

3.2 Swept-plane collision

The collision detector is the single most important piece of the interaction layer. A naïve bounding-sphere check would let a stick “hit” a drum just by entering its volume, without regard for direction of motion or position on the surface. The hi-hat problem is the clearest: its bounding sphere is tall enough that horizontal motion nearby would trigger false positives.

We instead use a *swept-plane* test per instrument. Let $\mathbf{c} \in R^3$ and $\hat{\mathbf{n}} \in R^3$ be the world-space centre and unit normal of the instrument’s playable surface, and r its radius (figure 2). Given the stick tip’s positions $\mathbf{p}_{t-\Delta t}$ and $\mathbf{p}_t$ on consecutive frames, compute the signed distances

$$d_{\text{prev}} = (\mathbf{p}_{t-\Delta t} - \mathbf{c}) \cdot \hat{\mathbf{n}}, \quad d_{\text{curr}} = (\mathbf{p}_t - \mathbf{c}) \cdot \hat{\mathbf{n}}.$$

A hit is registered iff $d_{\text{prev}} > 0$ and $d_{\text{curr}} \leq 0$: the tip *was* on the striking side of the plane last frame and has now crossed (or just touched) it. The contact point is obtained by linear interpolation,

$$\mathbf{p}_{\text{hit}} = \mathbf{p}_{t-\Delta t} + \frac{d_{\text{prev}}}{d_{\text{prev}} - d_{\text{curr}}}(\mathbf{p}_t - \mathbf{p}_{t-\Delta t}),$$

and the hit is only accepted if its radial distance in the plane, $\|(\mathbf{p}_{\text{hit}} - \mathbf{c}) - ((\mathbf{p}_{\text{hit}} - \mathbf{c}) \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}\|$, is at most r . The strike velocity is the component of tip motion along the normal,

$$v_n = \frac{d_{\text{prev}} - d_{\text{curr}}}{\Delta t},$$

normalised by the expected peak tip speed (2.5 m/s) and clamped to $[0, 1]$. Using v_n rather than raw speed means a stick moving sideways across a cymbal registers as softly as it should, while a direct perpendicular hit registers at full intensity.

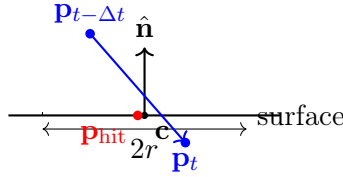


Figure 2: Swept-plane collision test. The segment $\mathbf{p}_{t-\Delta t} \rightarrow \mathbf{p}_t$ crosses the plane; the crossing point must lie within radius r of the centre for the hit to register.

To prevent re-triggering while the stick lingers against the surface, each controller maintains an **engaged** set of instruments it is currently in contact with. An instrument is removed from the set only when the tip has receded more than 2 cm back from the plane ($d_{\text{curr}} > 0.02$), at which point another crossing may fire a new hit. A 90 ms per-instrument cooldown guards against spurious double triggers.

Because the plane is defined in *instrument-local space* and transformed to world every frame with the instrument’s world matrix, the test is orientation-agnostic: cymbals tilted in two axes, rack toms tilted forward, or the bass drum facing horizontally toward the player all use the same code path.

3.3 Particle system

Hit feedback is reinforced by a pre-allocated ring buffer of 400 point-sprite particles, rendered with vertex colours and additive blending. A hit emits between 14 and 40 particles (scaled by velocity) at the contact point. Velocities are split into a normal component (along $\hat{\mathbf{n}}$, pushing the spray off the surface) and a tangential component distributed uniformly in the plane. The result is that cymbal hits spray sideways along the edge, while drum hits puff upward off the head. Gravity and a tangential drag term decelerate the particles into realistic arcs.

3.4 Internal audio synthesis

The headset-side audio path uses the Web Audio API. A master bus feeds a `DynamicsCompressorNode` and then `audioContext.destination`, giving predictable loudness. Each hit creates a fresh voice graph consisting of oscillators, filtered noise buffers, and a velocity-scaled gain, all routed through a `StereoPannerNode` whose pan value is the hit’s x coordinate clamped to $[-1, 1]$.

Synthesis recipes broadly match those in the Pd patch (section 5) but are implemented imperatively rather than as a dataflow graph. Their purpose is immediate, low-latency feedback in the headset — the Pd engine is the production path.

4 Bridge Server

The bridge (`bridge/index.js`) is 130 lines of Node.js. It runs Express for an `/status` JSON endpoint, Socket.IO for browser communication, and the `osc` package for Pure Data. Three event handlers cover the entire behaviour:

- `drumHit` from the VR client is broadcast to spectators (for visual mirroring) *and* translated into an OSC packet `/drum/hit` with arguments (`instrument`, `velocity`, `x`, `y`, `z`) sent to `127.0.0.1:8000`.
- `xrPose` is relayed to other clients so the desktop mirror can update its camera and ghost-controller meshes.
- `disconnect` of the VR client triggers an `xrSessionEnd` broadcast so spectators reset to their default view.

Pose data is sent every other frame (~ 36 Hz) via `socket.volatile.emit`, which drops messages rather than buffering them if the link saturates. This keeps spectator latency bounded even on flaky Wi-Fi.

5 Pure Data Patch and the Dataflow Paradigm

Pure Data is a *visual dataflow language* for real-time audio and multimedia. Programs are *patches* consisting of boxes (object invocations) connected by cords (edges) through which data flows. There are two fundamental rates: *message rate*, where events propagate eagerly on demand, and *audio rate*, where 64-sample vectors flow at a constant $44\,100/64 \approx 689$ Hz through objects whose names end in a tilde. The distinction is visible in the patch: audio cords are drawn thicker.

This project's patch, `pd/drumset.pd`, follows a classic three-stage dataflow pipeline: *input*, *voicing*, *mix*.

5.1 Input and dispatch

The top of the canvas demultiplexes incoming OSC:

```
[netreceive -u -b 8000]          ; UDP listener on port 8000
|
[oscparse]                      ; raw bytes -> typed list
|
[route /drum/hit]                ; drop any non-hit messages
|
[unpack s f f f f]              ; name, velocity, x, y, z
| | | | |
| | [s hit-x] [s hit-y] [s hit-z]
| |
+---+--> [pack s f] --> [route bassdrum snare hitom ...]
                        | | | | |
                        [s fire-bassdrum] ...
```

`[unpack]` fires its outlets right-to-left, which means the z , y , x , and velocity values are published to their respective `[s]` buses *before* the instrument symbol reaches `[route]`. By the time a voice's `[r fire-...]` fires, all context is already latched. This ordering guarantee is a hallmark of Pd's cold/hot inlet semantics and is the reason no explicit synchronisation primitives are required.

[pack s f] recombines the instrument name and velocity into a two-element list. When [route] matches a name, it strips that name and outputs the remaining list — a single float, the velocity — through the matched outlet. Each voice therefore receives the velocity through a single [r fire-<name>] receive, with no additional wiring needed.

5.2 Sample-based voicing

The production audio engine is *sample-based*, not synthesised. Each voice plays back one or more pre-recorded WAV files triggered by velocity. Two abstraction files implement the two voice flavours:

- **voice-multi.pd** — a **three-layer velocity-crossfaded** player used for the bass drum, snare, and hi-hat. Three recordings of the same drum at soft, medium, and hard intensity play *simultaneously*, each weighted by a factor derived from the incoming velocity so that the three weights sum to unity.
- **voice-single.pd** — a single-sample player used for the three toms and the two cymbals. One recording is scaled linearly by the incoming velocity.

Abstractions in Pd take creation-time arguments: inside **voice-multi.pd** every reference to \$1 is substituted at load time by the argument passed in the parent. Instantiating [voice-multi bassdrum] in the main patch therefore resolves to a voice that listens on [r fire-bassdrum] and reads the arrays **bassdrum-soft**, **bassdrum-med**, **bassdrum-hard**.

The velocity-to-weight mapping is a pair of tent functions and their midpoint complement (figure 3). For an incoming velocity $v \in [0, 1]$,

$$\begin{aligned} w_{\text{soft}}(v) &= \max(0, 1 - 2v), \\ w_{\text{med}}(v) &= \min(2v, 2(1 - v)), \\ w_{\text{hard}}(v) &= \max(0, 2v - 1). \end{aligned}$$

w_{soft} peaks at $v = 0$ and falls linearly to 0 at $v = 0.5$; w_{hard} is its mirror; w_{med} peaks at $v = 0.5$ and falls linearly to 0 at both ends. The three curves always sum to exactly 1, preserving loudness across the velocity range while the *timbre* morphs smoothly between the three recordings — the crucial property that distinguishes multi-sampling from simple gain-scaling. A real drum struck softly does not merely produce a quieter version of its hard hit; the spectral envelope changes too, and the three recordings capture exactly those changes.

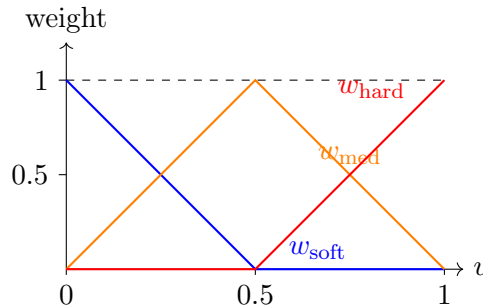


Figure 3: Three-layer velocity crossfade weights. The curves form a partition of unity over $[0, 1]$, so the combined loudness is constant while timbre morphs between the three recordings.

Sample sources

Fourteen WAV files live in `pd/samples/`. The multisample layers for bass/snare/hi-hat come from MPC-Tutor’s *8 Layer Drum Kit Tutorial* pack (the kit ships with eight MIDI-velocity layers; the patch uses layers 31, 79, and 127 for soft/medium/hard). The single-sample drums come from the *99Sounds 99 Drum Samples I* pack, one acoustic recording per drum. Both sources are distributed free of charge for non-commercial use.

Loading

At patch open, a single `[loadbang]` fires a chained message containing fourteen comma-separated `read -resize` commands at one `[soundfiler]` object. The receiving `[table name 100]` declarations provide the target arrays; `soundfiler`’s `-resize` flag expands each array to fit its source file. Load is synchronous and completes in well under a second for the 14-file set.

5.3 Master bus

All voices `throw~` to the shared buses `mix-L` and `mix-R`. These are audio-rate send/receive pairs; Pd sums the contributions automatically. At the bottom of the canvas, `catch~` objects retrieve the mixed signal, a master trim gain scales it, a `clip~ -0.98 0.98` pair provides a cheap soft limit against inter-sample peaks, and `dac~ 1 2` outputs to the first two channels of the audio interface.

5.4 Why dataflow matters here

The patch is *textually* larger than the equivalent Web Audio code but *conceptually* smaller: there is no sequencing, no ordering of function calls, no shared mutable state to reason about. Each voice is an island whose wiring visibly constrains what it can do. Editing the patch while it is playing is safe — changes to a message box or a filter frequency take effect on the next sample block, with no restart. This liveness is one of the two qualities that distinguish dataflow languages for multimedia from imperative ones.

6 Spectator Mirror

A non-trivial part of the system is that the desktop browser renders a live *mirror* of the VR session, useful for demos, recording, and debugging without constantly swapping the headset.

The mechanism is shared-codebase: both clients load the same `main.js`. A boolean `isSpectator` starts `true` and flips on `renderer.xr.addEventListener('sessionstart', ...)`. While the VR client is presenting, it publishes a compact pose message every other frame:

```
socket.volatile.emit('xrPose', {
  cam: { px, py, pz, qx, qy, qz, qw },    // headset pose
  ctrl: [ctrl0Pose, ctrl1Pose]           // controller poses
});
```

The spectator copies those values directly into its `THREE.PerspectiveCamera` and two translucent “ghost” drumstick meshes, so it sees the VR user’s viewpoint plus their hand positions. Drum hits are mirrored too: each hit triggers a `drumHitFeedback` broadcast that the spectator uses to replay the visual flash, head flex, and particle burst — but deliberately not the sound, which stays on the VR side.

7 Building and Running

7.1 Prerequisites

- Node.js 18 or newer on the host PC.
- Pure Data vanilla ≥ 0.50 (no externals required).
- A Pico 4 headset on the same Wi-Fi network as the PC, with Guardian/boundary configured and the browser set to allow WebXR.

7.2 One-time setup

```
npm run install:all    # installs root + frontend + bridge deps
```

7.3 Running the system

1. `npm start` in the repo root launches Vite (HTTPS :5173) and the bridge (:3000) concurrently.
2. Open `pd/drumset.pd` in Pure Data and enable DSP (Ctrl-/). If the bridge is already running, the terminal should show `[OSC] Ready -> 127.0.0.1:8000`; if Pd is now listening on 8000, hits will be synthesised.
3. On the PC, open `https://localhost:5173` and accept the self-signed certificate. This window is the spectator.
4. On the Pico, open `https://<PC-IP>:5173`, accept the certificate, and press “Enter VR”. Play.

7.4 Verification

`http://localhost:3000/status` returns a small JSON object describing whether a VR session is connected, the number of Socket.IO clients, and the OSC target. A quick end-to-end sanity check is to press a controller trigger without aiming: it should produce an audible hit from both the headset (Web Audio) and the PC speakers (Pd), and a matching visual flash in the spectator window.

8 Performance Notes

The target is a stable 72Hz on the Pico XR2 Gen 1. Choices made with that budget in mind:

- The entire scene has $\approx 2\,500$ triangles and ≈ 30 draw calls. Environment IBL is pre-rendered once at startup so it imposes no per-frame cost.
- Particles are a single 400-element ring buffer drawn as one `THREE.Points` object, not as per-particle meshes.
- `Socket.IO` pose messages are emitted as `volatile`, meaning the client drops rather than queues them if the link is congested. This avoids head-tracking stutter cascading into network latency.
- The environment map and the platform are omitted during VR presentation (`scene.background = null`) both for passthrough compatibility on the Pico browser and to eliminate a full-screen fill.

9 Limitations and Future Work

Fully multisampled toms and cymbals. The bass drum, snare, and hi-hat are three-layer velocity-crossfaded; the three toms and two cymbals are single-sample. Bringing the latter group up to three-layer multisampling would complete the engine, but requires a multisampled kit that covers toms and cymbals — harder to source than multisampled shells alone. The patch is structured to make this a one-line change per voice: replacing `[voice-single hitom]` with `[voice-multi hitom]` and dropping three new WAV files into `pd/samples/` is all that is needed.

Convolution reverb. A convolution impulse response loaded into `[partconv~]` on the master bus would add realistic room ambience. The current master chain ends in a soft clip; slotting a wet/dry reverb stage between the catch and the limiter is mechanical.

Hand tracking. The Pico 4 supports hand tracking, but the current frontend only consumes controller input. Extending `tickCollision` to additionally sweep from finger-tip joints would let users play without holding the controllers.

Physical cymbal dynamics. Cymbals already wobble angularly on hit via a spring-damped model (spring constant $k = 22$, damping $c = 3.5$). The drum shells, by contrast, only apply an axial compression. A displacement-mapped drum-head shader that ripples outwards from the contact point would add another order of visual realism for negligible GPU cost.

Haptics refinement. Current haptic feedback is a single `pulse(intensity, ms)` call on the gamepad’s haptic actuator. A velocity-shaped pulse (e.g. a short burst followed by a slower decay for bass hits) would better convey stick weight.

10 Conclusion

MR Drumset composes three very different programming paradigms into a single interactive instrument: imperative 3D scene-graph code on the frontend, event-driven JavaScript on the Node bridge, and visual-dataflow programming in Pure Data. The collision layer demonstrates that correctness in VR interaction comes from modelling surfaces as proper geometric entities — planes with centres, normals, and radii — not as bounding-sphere approximations. The Pd patch demonstrates, in turn, that an entire audio engine can be expressed as a small, readable graph whose structure *is* its documentation.

References

- Three.js documentation. <https://threejs.org/docs/>
- WebXR Device API. W3C Recommendation. <https://www.w3.org/TR/webxr/>
- M. Puckette. *Pure Data*. In Proc. International Computer Music Conference, 1996.
- M. Puckette. *The Theory and Technique of Electronic Music*. World Scientific, 2007. <http://msp.ucsd.edu/techniques.htm>
- Web Audio API. W3C Recommendation. <https://www.w3.org/TR/webaudio/>
- Open Sound Control 1.0 specification. https://opensoundcontrol.stanford.edu/spec-1_0.html